



FLOWDIARY

AI-Powered Ethical Hacking

Lecture 9 — Status Codes, Cookies, Sessions & Authentication

Anatomy of a request & response · methods · headers · query strings vs body · content types · HTTPS / TLS basics

Tutor: Engr. Ibrahim Auwal

Why this lecture matters for ethical hackers

Lecture 8 showed you the anatomy of HTTP messages. This lecture zooms in on the part attackers care about most: how a website remembers who you are. HTTP itself is stateless — it forgets you the instant a response is sent — so every site rebuilds a sense of identity using status codes, cookies, sessions, and tokens.

Authentication is the front door of every application. Broken authentication and broken session management appear on every list of top web risks. If you understand exactly how identity is created, carried, and trusted, you understand where it can be stolen, forged, or bypassed.

Learning Objectives

By the end of this lecture you should be able to:

- Interpret each HTTP status code family and the key individual codes.
- Explain what cookies are, how they are set, and what their security flags do.
- Describe how server-side sessions work and how they relate to cookies.
- Distinguish session-based auth from token-based auth.
- Explain the structure of a JWT and its common weaknesses.
- Trace a typical login flow and identify where it can be attacked.

1. Status Code Families in Depth

You met status codes briefly in Lectures 6 and 8. Because they are how a server signals the outcome of every request, an ethical hacker reads them constantly — they reveal what exists, what is protected, and what is breaking. The first digit sets the category.

Family	Category	What it signals to a tester
1xx	Informational	Rarely seen; request received, still processing.
2xx	Success	The action worked — the resource exists and is accessible.
3xx	Redirection	Content moved; follow-the-trail clues about site structure.
4xx	Client error	You asked wrongly — often the most informative range.
5xx	Server error	The server broke — may leak internal detail.

1.1 The codes that matter most in security work

Code	Name	Why a hacker cares
200	OK	Resource exists and returned content.
301 / 302	Redirect	Maps site structure; watch open redirects.
401	Unauthorized	Auth required — you are not logged in.
403	Forbidden	Exists but blocked — a target for bypass.
404	Not Found	Nothing here (or hidden to look absent).
429	Too Many Requests	Rate limiting is active — affects brute-forcing.
500	Server Error	Something broke; may leak stack traces.

Security lens: 401 vs 403 vs 404

The difference is a goldmine. A 401 says "log in first." A 403 says "you're logged in but not allowed" — confirming the resource exists, which invites privilege-escalation testing. A 404 says "nothing here."

Enumeration trick: if /admin returns 403 while /random returns 404, you have just confirmed an admin panel exists — even without access. Inconsistent codes leak the shape of the app.

2. Cookies — How the Web Remembers You

HTTP is stateless: each request stands alone with no memory of the last. Cookies are the fix. A cookie is a small piece of data the server asks the browser to store and send back on every future request to that site — giving the illusion of continuity.

2.1 How a cookie is set and returned

The server sets a cookie with a Set-Cookie response header. The browser stores it and automatically attaches it, using the Cookie header, on every subsequent request to that domain — as you saw in the raw messages in Lecture 8.

Setting and sending a cookie

Response from server:

```
Set-Cookie: session=8f3a91b2; HttpOnly; Secure; SameSite=Strict
```

Every later request from the browser:

```
Cookie: session=8f3a91b2
```

2.2 Cookie security flags

Flags on a cookie control how safely it behaves. Checking these is routine during an assessment.

Flag	What it does	Why it matters
HttpOnly	Hides cookie from JavaScript	Blocks XSS from stealing the session.
Secure	Only sent over HTTPS	Stops leakage over plain HTTP.
SameSite	Limits cross-site sending	Defends against CSRF.
Expires / Max-Age	Sets a lifetime	Short life shrinks the theft window.
Domain / Path	Scopes where it is sent	Too-broad scope over-shares the cookie.

Security lens: the cookie IS the identity

In most web apps, whoever holds the session cookie is treated as that user — no password re-check. This makes cookie theft a complete account takeover.

Attack paths: stealing cookies via XSS (defeated by HttpOnly), sniffing them over HTTP (defeated by Secure), riding them cross-site via CSRF (defeated by SameSite), or fixing a known value before login (session fixation).

3. Sessions — Identity Stored on the Server

A cookie by itself is just a small value. The most common way to use it is the session model, where the real data lives on the server and the cookie only carries a reference to it.

3.1 How server-side sessions work

1. **You log in.** The server verifies your credentials once.
2. **The server creates a session.** It stores your details (user ID, roles) in its own memory or database, under a random session ID.

3. **The ID goes to you as a cookie.** Only the meaningless random ID travels to the browser — never your actual data.
4. **Each request carries the ID.** The server looks it up to remember who you are.
5. **Logout / expiry destroys the session.** The server discards the stored record, and the ID becomes worthless.

The key property: the browser holds only a random pointer. All the trusted information stays server-side, where the user cannot tamper with it.

Security lens: attacking sessions

Predictable session IDs: if IDs are guessable or sequential, an attacker can forge a valid one. They must be long and random.

Session fixation: if the ID is not regenerated at login, an attacker who planted it earlier can ride the authenticated session.

No expiry / no logout: sessions that never die give stolen IDs unlimited life.

4. Tokens — Identity Carried by the Client

Sessions keep state on the server, which is simple but harder to scale across many servers and awkward for APIs and mobile apps. The token model flips this: instead of storing identity on the server, the server hands the client a signed token that proves who they are, and the client presents it on each request.

4.1 Sessions vs tokens

	Session (stateful)	Token (stateless)
Where identity lives	On the server	In the token, held by client
Server stores state?	Yes, per session	No — token is self-contained
Sent how?	Session-ID cookie	Often Authorization header
Revoking access	Easy — delete server record	Harder — token valid until expiry
Good for	Classic web apps	APIs, mobile, microservices

Tokens are commonly sent in the Authorization header you saw in Lecture 8, typically as a "Bearer" token — meaning whoever bears it is granted access.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...
```

5. JWTs — JSON Web Tokens

The most common token format is the JWT (JSON Web Token). It is a self-contained, signed packet of identity information. Understanding its three parts is essential, because JWT misuse is a frequent, high-impact vulnerability.

5.1 The three parts of a JWT

A JWT is three Base64-encoded sections joined by dots: header.payload.signature.

Structure of a JWT

```
eyJhbGciOiJIUzI1NiJ9 . eyJ1c2VyIjoiaWJyYWVhpbSJ9 . SflKxwRJSMeKKF2QT4
    HEADER          PAYLOAD          SIGNATURE
```

- **Header:** states the token type and the signing algorithm (e.g. HS256).
- **Payload:** the claims — data like user ID, role, and expiry time.
- **Signature:** a cryptographic seal over the header and payload, proving they were not altered.

Critical point: encoded is NOT encrypted

A standard JWT payload is only Base64-encoded, not encrypted. Anyone can decode and read it. Never put secrets (passwords, sensitive data) in a JWT payload — assume the user can read every claim.

The signature is what protects it. It does not hide the data; it proves the data was not changed. Reading is open; tampering is what the signature detects.

5.2 Common JWT vulnerabilities

- **alg: none attack:** if the server accepts a token declaring no algorithm, an attacker strips the signature and forges any claims.
- **Weak signing secret:** a short or guessable HMAC secret can be cracked offline, letting an attacker mint valid tokens.
- **No expiry check:** if the server ignores the expiry claim, stolen tokens live forever.
- **Claim tampering:** changing "role": "user" to "role": "admin" — harmless only if the signature is properly verified.

6. How Web Authentication Typically Works

Now we assemble the pieces into the login flow you meet on nearly every site. Watch how each earlier concept plays its part.

6. **Registration.** You create an account. The server stores your password as a salted hash — never in plain text.
7. **Login request.** You POST your credentials in the request body (Lecture 8), over HTTPS so they are encrypted in transit.
8. **Verification.** The server hashes what you sent and compares it to the stored hash. A match means success.
9. **Identity issued.** The server returns either a session-ID cookie (session model) or a signed token/JWT (token model).
10. **Authenticated requests.** Your browser automatically sends the cookie, or your app sends the token, on every following request.
11. **Authorization checks.** For each request the server confirms not just who you are (authentication) but what you may do (authorization).
12. **Logout / expiry.** The session is destroyed or the token expires, ending the identity.

Authentication vs authorization — do not confuse them

Authentication = who are you? Proving identity (login).

Authorization = what are you allowed to do? Enforcing permissions after identity is known.

Most access-control bugs are authorization failures: the app knows who you are but fails to check whether you should reach a given record or action (the IDOR idea from Lecture 7).

Security lens: where login flows break

Weak passwords / no rate limiting → brute-force and credential stuffing (watch for 429s).

Plain-text or weakly hashed passwords → catastrophic on database breach.

Username enumeration → different responses for valid vs invalid users leak accounts.

Insecure session/token handling → everything from Sections 2–5.

Missing MFA → a single stolen password is game over.

7. Recap and Key Takeaways

- **Status code families** signal outcomes; 401/403/404 differences leak the shape of an app.
- **Cookies give stateless HTTP a memory;** their security flags (HttpOnly, Secure, SameSite) are routine checks.
- **Sessions keep identity on the server** and hand the client only a random reference.
- **Tokens keep identity in the client** as a signed, self-contained credential — great for APIs, harder to revoke.
- **JWTs are encoded, not encrypted;** the signature proves integrity, and misuse (alg:none, weak secrets) is a common bug.

- **Authentication proves who you are; authorization decides what you can do** — and the two must both be enforced server-side.

Discussion & Lab Prompts

1. Log into a test site and inspect the Set-Cookie header. Which security flags are present? Which are missing, and what is the risk?
2. Take a sample JWT and decode its header and payload (e.g. on jwt.io). What claims can you read? Why is that safe or unsafe?
3. Request /admin (or similar) while logged out. Do you get 401, 403, or 404 — and what does that tell you about the app?
4. Compare session vs token auth for a mobile app backend. Which would you choose and why?
5. Discuss: an attacker steals a valid session cookie. Walk through what they can do and which single flag would have stopped the theft.

Key Terms Glossary

Term	Quick definition
Stateless	HTTP forgets you between requests.
Cookie	Small data the browser stores and resends.
Set-Cookie	Response header that plants a cookie.
Session	Server-side identity keyed by a random ID.
Session fixation	Reusing a known session ID to hijack a login.
Token	Self-contained credential held by the client.
JWT	Signed JSON token: header.payload.signature.
Bearer token	Whoever holds the token gets access.
Authentication	Proving who you are.
Authorization	Deciding what you may do.